Assignment 4: Machine Translation with LSTM, GRU and Encoder–Decoder Models

Course: CS4101 | CS6103

Deadline: Monday, 9 November, 2026

Instructions

- Work in groups of three, with the same members as Assignment 1.
- Write properly indented, commented, and well-structured code.
- Plagiarism results in zero for all involved group members.

Dataset

- Use the IIT Bombay English–Hindi parallel corpus to translate English into Hindi: https://huggingface.co/datasets/cfilt/iitb-english-hindi
- The corpus is large, noisy, and contains many duplicate English sentences. Work with a fixed-seed subset of at most 20,000 pairs having 1 to 10 tokens on both sides, and report the split sizes, vocabulary sizes, and unknown-token rate per language.
- Choose and justify a tokenization for Hindi (e.g., handling of Hindi sentence-ending punctuation), and apply it consistently for training and BLEU.

Task 1: Recurrent Cells and Encoder–Decoder Translation

- Implement LSTM and GRU cells from scratch (basic tensor operations only) that handle padded batches with true sequence lengths, and document your initialization.
- Verify your cells against PyTorch's built-in layers.
- Report the parameter count of each cell for $d = 128$, $H = 256$, and explain the LSTM/GRU ratio.
- Compare a vanilla RNN, your LSTM, and your GRU for $T \in \{10, 25, 50, 100\}$ ($B = 16$, $d = 32$, $H = 64$, inputs $\sim \mathcal{N}(0, I)$): plot, on a log scale, the mean norm of the gradient of $\|\mathbf{h}_T\|_2^2$ with respect to the first input, averaged over seeds $0, \ldots, 4$, and explain what you observe.
- Build a configurable encoder–decoder translation model from your cells, trained with teacher forcing and decoded greedily (at most 20 steps).
- Explain how teacher forcing differs from free-running decoding and what problem this causes, illustrating with a Hindi example from your outputs.
- Train for at most 10 epochs, tracking loss, perplexity, token accuracy, and validation BLEU-4 after each epoch.
- Implement corpus-level BLEU-4 yourself.
- Compare it with `sacrebleu` on the same pre-tokenized Hindi translations, and use `sacrebleu` for model selection.
- Discuss why BLEU may under-reward valid Hindi translations (e.g., free word order).

Task 2: Cell Type and Hidden Size

- Train LSTM and GRU models with hidden sizes 64 and 128 (four configurations), keeping all other hyperparameters fixed and documented. Take each run's checkpoint as its earliest epoch with the maximum validation BLEU-4 (`sacrebleu` score).
- Report parameters, training time, checkpoint metrics, and training curves for every run.
- Discuss the effect of cell type and hidden size on quality, cost, and underfitting/overfitting, relating it to Hindi vocabulary size and unknown-token rate.
- Justify why validation BLEU rather than validation loss is (or is not) the better checkpoint criterion, using your curves.

Task 3: Final Model

- Select the final model by validation BLEU-4 (`sacrebleu`; ties: fewer parameters, then the order LSTM-64, GRU-64, LSTM-128, GRU-128).
- Evaluate it once on the test set (both BLEU-4 scores, loss, perplexity, token accuracy).
- Show the first 10 test translations.
- Analyze at least three erroneous translations by error type (e.g., word order, gender/number agreement, postpositions, unknown words).
- Discuss how English–Hindi word-order differences (SVO versus SOV) interact with a fixed-size encoder state, using your translations as evidence.
- Justify your selection.
- Argue whether the ranking of your models would hold under a different random seed.

Restrictions

- Built-in recurrent layers (e.g., LSTM, GRU) are allowed only for the Task 1 verification.
- The encoder–decoder, training loop, and decoding must be implemented by your group.
- Only standard Python libraries, numpy, pandas, matplotlib, seaborn, and PyTorch are permitted.
- `sacrebleu` may be used for model selection and for comparison with your own BLEU implementation. You must implement BLEU-4 independently.
- Attention, Transformers, pretrained models, and Hugging Face libraries are not allowed.

Reproducibility and Submission

- Random seeds must be explicitly set everywhere randomness is involved, including data subset selection, parameter initialization, shuffling, and the gradient-flow experiment.
- If a GPU is used, enable deterministic operations or report any remaining nondeterminism.
- The notebook must run end to end and reproduce all results reported in the PDF.
- Submit one ZIP file containing exactly one implementation notebook and one PDF report (methodology, results, analysis, and contributions of each member), named using the roll numbers of all group members (e.g., `rollno1_rollno2_rollno3.zip`), and upload it to https://forms.gle/uC1GRTV9H7ckAzeN9 by the deadline.